

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

Código de actividad: TA-IND-04

Trabajo Autónomo Individual — Informe Técnico

Análisis de Rendimiento Paralelo aplicado al Proyecto Fin de Curso

Del ordenamiento y *Top-N* distribuido en PySpark a la propuesta de integración
en el Sistema de Control de Laboratorios Informáticos (SCLI)

Asignatura: Aplicaciones Distribuidas (ISR-701)
Unidad: Unidad 4 — Cómputo Paralelo y Distribuido
Práctica de origen: GA-SUM-05 / PE-U4 — Procesamiento Distribuido con Apache Spark
Estudiante: Iván Andrés Villamarín Cuenca
Equipo de PE-U4: Equipo C — Freddy Farinango, Jeremy Gaibor, Iván Villamarín
PFC de referencia: SCLI — Sistema de Control de Laboratorios Informáticos
Transformación foco: T5 — Ordenamiento y Top-20
Repositorio (este informe): <https://github.com/iavillamarin98-pred/ta-ind-04-villamarin>
Repositorio de origen (PE-U4): <https://github.com/ffarinangog2/pe-u4-spark-equipo-c>
Commit de los datos base: 3a0ae757b6f8288bf095905e286bca75d8026a36
Docente: Gleiston C. Guerrero-Ulloa, M.Sc.
Período académico: 2026–2027 PPA
Fecha: 9 de agosto de 2026

Análisis de Rendimiento Paralelo aplicado al Proyecto Fin de Curso: Del ordenamiento y *Top-N* distribuido en PySpark a la propuesta de integración en el Sistema de Control de Laboratorios Informáticos (SCLI)

Iván Andrés Villamarín Cuenca
Facultad de Ciencias de la Computación — UTEQ

Resumen—Este informe analiza de forma individual los resultados de *speedup* obtenidos en la práctica grupal GA-SUM-05/PE-U4 — *Procesamiento Distribuido con Apache Spark* — para la transformación T5 (ordenamiento global y selección de las 20 solicitudes de mayor demanda) sobre un conjunto sintético de 500 000 solicitudes de reserva de laboratorios, ejecutada en pandas y en PySpark 3.5.0 sobre un clúster Spark Standalone real de hasta cuatro *executors*. A diferencia de otras transformaciones del equipo, T5 cuenta con mediciones propias de escalado en $N = 1, 2, 4$ *executors*, lo que permite un análisis cuantitativo completo frente a la Ley de Amdahl y la métrica de Karp-Flatt. Se determina que, a diferencia de las transformaciones más ligeras del *pipeline*, el umbral de rentabilidad de T5 ya se cruza al volumen actual, y se propone un punto concreto de integración de un servicio de generación de rankings en la arquitectura de microservicios del Sistema de Control de Laboratorios Informáticos (SCLI).

I. INTRODUCCIÓN Y CONTEXTO

El presente documento constituye la entrega individual TA-IND-04 de la asignatura Aplicaciones Distribuidas (ISR-701), Unidad 4 – Cómputo Paralelo y Distribuido. Su objetivo es explotar intelectualmente, de forma individual, los datos experimentales producidos en equipo durante la práctica GA-SUM-05/PE-U4 — *Procesamiento Distribuido con Apache Spark: comprobación experimental de la Ley de Amdahl*, en la cual se compararon cinco transformaciones (T1–T5) sobre un mismo conjunto de datos, ejecutadas alternativamente con pandas (motor secuencial de referencia) y con PySpark (motor distribuido) sobre un clúster Spark Standalone real.

El experimento de origen se realizó sobre un conjunto sintético y reproducible de 500 000 solicitudes de reserva de laboratorios y 40 laboratorios, relacionados 1:N mediante `laboratorio_id`, generado con semilla fija 20260804 para el dominio FUVV – Sistema de Control de Laboratorios Informáticos. Se utilizan datos sintéticos porque no existen 500 000 registros operacionales públicos adecuados para el volumen exigido y porque así se evita el uso de información personal real de estudiantes o docentes.

Los datos base de este informe proceden del repositorio del equipo:

- **Repositorio PE-U4:** <https://github.com/ffarinangog2/pe-u4-spark-equipo-c>
- **Commit exacto:** 3a0ae757b6f8288bf0959059-05e286bca75d8026a36 (rama main)
- **Archivo fuente:** resultados/tiempos_resumen.csv y resultados/tiempos_crudos.csv

La transformación declarada como foco individual de este análisis es **T5 – ordenamiento global y selección *Top-20***: las solicitudes se ordenan de forma estable por número de participantes y duración descendentes, con fecha e identificador como criterios de desempate ascendentes, y se seleccionan las 20 de mayor demanda. Es, junto con T3, una de las transformaciones para las que el equipo generó mediciones escaladas propias en $N = 1, 2, 4$ *executors*, condición indispensable para el análisis cuantitativo de escalabilidad que exige este instrumento. El dominio del PFC de referencia es el Sistema de Control de Laboratorios Informáticos (SCLI), un sistema de microservicios (Spring Boot, CockroachDB, Docker Compose) actualmente en desarrollo para automatizar la gestión de usuarios, laboratorios, reservas y solicitudes de la institución.

II. ANÁLISIS PROPIO DE LOS RESULTADOS DE *speedup*

II-A. Datos base y trazabilidad

La Tabla I reproduce, con los valores exactos tomados del commit declarado, las medianas de cinco repeticiones para las cinco transformaciones evaluadas por el equipo, junto con el escalado de T5 en $N = 1, 2, 4$ *executors*. La ejecución PySpark se realizó sobre Spark Standalone (`spark://127.0.0.1:7077`), PySpark 3.5.0, con un núcleo y 512 MiB de memoria por *executor*, sesión independiente creada y destruida para cada configuración de N (sin *caching* entre configuraciones), y con `spark.sql.shuffle.partitions` y

`spark.default.parallelism` fijados explícitamente en N — condición declarada en la configuración del equipo (verificada mediante `StatusTracker` antes de cada medición) que resulta determinante para el diagnóstico de la Sección III.

Tabla I: Tiempos medianos (5 repeticiones) por transformación y configuración. Fuente: resultados/tiempos_resumen.csv, commit 3a0ae757.

Transf.	Motor / config.	Mediana (s)
T1	pandas	0,2119
T1	PySpark, $N=4$	0,3457
T2	pandas	0,1171
T2	PySpark, $N=4$	0,6875
T3	pandas	0,2889
T3	PySpark, $N=4$	0,8657
T4	pandas	2,2659
T4	PySpark, $N=4$	1,8100
T5	pandas	1,5252
T5	PySpark, $N=1$	1,5632
T5	PySpark, $N=2$	1,1733
T5	PySpark, $N=4$	1,3565

De las cinco transformaciones, `pandas` superó a `PySpark` en T1, T2 y T3, mientras que `PySpark` fue más rápido en T4 (columna derivada, con cómputo por fila más costoso) y **T5** (ordenamiento total y *top-N*), con un *speedup* cruzado $S_{\text{pandas/PySpark}} = T_{\text{pandas}}/T_{\text{PySpark}}(N=4) = 1,5252/1,3565 = 1,1244$. Este patrón es consistente con la literatura: `PySpark` amortiza mejor su sobrecarga de planificación cuando el costo de cómputo por registro es alto — el ordenamiento global de 500 000 filas antes de recortar el *top-20* es, junto con T4, la operación más costosa del *pipeline* en `pandas` (1,5252 s) — mientras que en operaciones ligeras (filtrado, agrupación simple, *join* sencillo) esa sobrecarga fija domina el tiempo total [1].

II-B. Contraste experimental frente a la predicción de Amdahl

A diferencia del *speedup* cruzado `pandas/PySpark` reportado arriba, el análisis de escalabilidad exige el *speedup* interno de `PySpark`, tomando como referencia $N = 1$:

$$S(N) = \frac{T_{\text{PySpark}}(N=1)}{T_{\text{PySpark}}(N)}, \quad N \in \{1, 2, 4\}. \quad (1)$$

Estas son magnitudes distintas y no deben confundirse: $S_{\text{pandas/PySpark}}$ compara dos motores a igual N , mientras que $S(N)$ de la Ec. (1) mide la ganancia de paralelizar dentro de `PySpark` al variar N , que es la que exige la Ley de Amdahl.

Con los tiempos de la Tabla I: $S(1) = 1,0000$, $S(2) = 1,3324$, $S(4) = 1,1524$. El resultado es, igual que en la transformación T3 del equipo, **no monótono**: el *speedup* en $N = 4$ es menor que en $N = 2$, es decir, añadir dos *executors* adicionales empeoró el rendimiento respecto de $N = 2$. Este comportamiento por sí solo ya adelanta que el modelo de Amdahl, que asume una fracción paralelizable constante y sobrecarga nula [2], no podrá ajustar simultáneamente los tres puntos.

Siguiendo la Ec. (2), formulación original de la Ley de Amdahl [3], se estimó la fracción paralelizable p mediante un ajuste de un único punto anclado en $N = 4$ (la configuración con mayor paralelismo disponible), despejando p de $1/S(4) = (1 - p) + p/4$:

$$p = \frac{1 - 1/S(4)}{0,75} = 0,1764.$$

Con este p , la Ec. (2) predice $S_{\text{teo}}(2) = 1,0967$ y, por construcción, $S_{\text{teo}}(4) = 1,1524$ (coincide exactamente con el punto de anclaje). El límite máximo teórico es $S_{\text{máx}} = 1/(1 - p) = 1,2141$: incluso con *executors* infinitos, T5 nunca podría acelerarse más de un 21,4 % respecto de un solo *executor* — una fracción paralelizable mucho más pequeña que la de T3 (donde $p \approx 0,4855$), coherente con que un ordenamiento global (*sort*) es intrínsecamente más dependiente de coordinación entre particiones que un *join*.

La desviación relativa (Sección 4.1 de la guía) se calcula como

$$\Delta = \frac{S_{\text{med}} - S_{\text{teo}}}{S_{\text{teo}}} \times 100 \, \%.$$

En $N = 4$, $\Delta = 0,0 \, \%$ por construcción del ajuste. En $N = 2$, sin embargo, $\Delta = +21,5 \, \%$: el *speedup* medido supera la predicción teórica, un caso de *speedup superlineal* local que la guía no enumera entre sus tres respuestas típicas (coincidencia, déficit, o $S < 1$) pero que es perfectamente consistente con el marco de Amdahl si se explica el mecanismo.

¿Coinciden los resultados con la predicción de Amdahl?

No de forma uniforme, y la razón tiene dos componentes identificables. Primero, el ajuste de p con un solo punto ($N = 4$) no es un ajuste por mínimos cuadrados sobre las tres observaciones; al anclarse en el extremo de mayor N , el modelo resultante no está obligado a explicar el comportamiento en $N = 2$, y de hecho no lo hace. Segundo, y más relevante en términos de sistema real: en $N = 2$ el volumen de trabajo por *executor* es el doble que en $N = 4$ (250 000 vs. 125 000 registros), lo que favorece una mejor razón entre trabajo útil y sobrecarga fija de *shuffle* y planificación por tarea; en $N = 4$ esa misma sobrecarga fija se paga cuatro veces por una fracción de datos cada vez menor por *executor*, un efecto de rendimientos decrecientes que Amdahl, al asumir sobrecarga nula, no puede capturar [2]. Esto también es coherente con el argumento de Gustafson de que el comportamiento del *speedup* depende fuertemente de cómo escala el trabajo por unidad de procesamiento, no solo del número de unidades [4].

II-C. Figura propia

La Fig. 1 (elaboración propia, generada con el script `figuras/generar_fig_speedup.py` incluido en este repositorio) superpone la curva teórica de Amdahl ajustada en $N = 4$ con los tres puntos experimentales de T5. La brecha visual entre la curva y el punto $N = 2$ hace explícita la superlinealidad discutida arriba, mientras que en $N = 4$ la curva y el punto coinciden por construcción del ajuste. Nótese la escala vertical mucho más comprimida que la de una transformación con mayor fracción paralelizable: el margen de

mejora total de T5 mediante paralelismo puro es, según $S_{\text{máx}}$, inferior al 22 %.

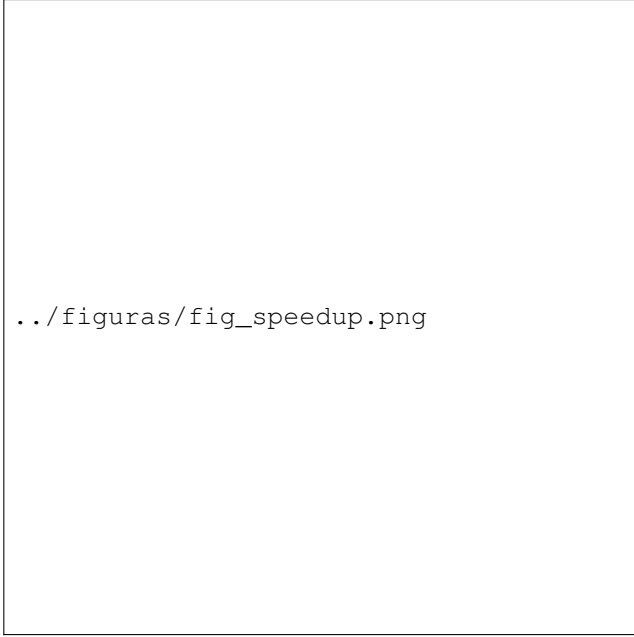


Figura 1: Speedup experimental $S(N)$ de T5 (ordenamiento y *Top-20*) frente a la curva teórica de Amdahl ajustada en $N = 4$ ($p = 0,1764$). Elaboración propia a partir de `tiempos_resumen.csv`, commit 3a0ae757.

III. DIAGNÓSTICO DE LA FRACCIÓN NO ESCALABLE

III-A. Métrica de Karp-Flatt y su tendencia

La fracción serial experimental e se calculó con la Ec. (3) para $N = 2$ y $N = 4$, usando los mismos valores de $S(N)$ obtenidos en la Sección II:

$$e(2) = \frac{1/S(2) - 1/2}{1 - 1/2} = 0,5010, \quad e(4) = \frac{1/S(4) - 1/4}{1 - 1/4} = 0,8236$$

La tendencia es inequívoca y creciente: e pasa de 50,1 % a 82,4 % entre $N = 2$ y $N = 4$. Según el criterio de interpretación de Karp y Flatt [5], una fracción serial que permanece aproximadamente constante al variar N indica un límite algorítmico de paralelismo; una fracción serial que *crece* con N , como en este caso, indica que la pérdida de eficiencia proviene predominantemente de la **sobrecarga creciente del motor**, no solo del algoritmo. A diferencia de T3, sin embargo, aquí el punto de partida ya es alto ($e(2) = 0,5010$): más de la mitad del tiempo de T5 con dos *executors* ya es no paralelizable, coherente con el bajo $p = 0,1764$ estimado en la Sección II — el ordenamiento global tiene, por su propia naturaleza algorítmica, una fracción serial estructuralmente mayor que un *join*.

III-B. Atribución de la sobrecarga a mecanismos concretos

Se identifican tres mecanismos concretos, coherentes con la arquitectura de ejecución de Spark [1], [6] y con la configuración efectivamente empleada por el equipo (Tabla de

configuración del repositorio de origen), que explican tanto el nivel alto de e como su crecimiento entre $N = 2$ y $N = 4$ para un ordenamiento global de 500 000 filas:

- **Range partitioning para el orden total.** A diferencia de una transformación estrecha, un `orderBy` global en Spark exige primero muestrear la distribución de las claves de ordenamiento (participantes y duración) para determinar los límites de rango de cada partición, y luego redistribuir — vía *shuffle* — cada fila a la partición cuyo rango le corresponde, de modo que el orden final quede correcto entre particiones. Con `spark.sql.shuffle.partitions = N` fijado por el equipo, al pasar de $N = 2$ a $N = 4$ este muestreo y la redistribución se fragmentan en el doble de particiones, cada una más pequeña, sin que el costo fijo de calcular los límites de rango y coordinar la redistribución se reduzca proporcionalmente — el mecanismo de rendimientos decrecientes que Amdahl, al asumir sobrecarga nula, no puede capturar [2].
- **Recolección final del Top-20 en el driver.** A diferencia de T3 (cuyo resultado son las 500 000 filas del *join*), T5 termina con una acción `limit(20)` que exige fusionar los resultados parcialmente ordenados de cada partición en un único orden global de solo 20 filas en el *driver*. Esa fusión final es inherentemente serial — es, de hecho, la porción de la fracción serial algorítmica genuina de T5, distinta de la sobrecarga del motor — y explica por qué p es estructuralmente bajo para esta transformación incluso antes de considerar la sobrecarga de *shuffle*.
- **Contención de recursos entre executors concurrentes en una sola máquina física.** Cada *executor* se configuró con un solo núcleo y 512 MiB de memoria, ejecutado como proceso JVM independiente sobre el mismo *host* físico (Spark Standalone local). Con $N = 4$ *executors* concurrentes compitiendo por la misma CPU, memoria y E/S de disco de una sola máquina, la presión de memoria durante el *shuffle* de ordenamiento se agrava de forma no lineal respecto de $N = 2$, un efecto que un clúster de nodos físicos separados no presentaría con la misma magnitud.

¿Qué parte de la no escalabilidad es atribuible al algoritmo y qué parte al motor? A diferencia de T3, en T5 ambas partes son sustanciales: la fusión final del *Top-20* en el *driver* es una fracción serial *algorítmica* genuina (no desaparece aunque el motor no tuviera sobrecarga alguna), mientras que el crecimiento de e entre $N = 2$ y $N = 4$ — que un algoritmo de ordenamiento intrínsecamente no escalable no explicaría por sí solo, porque su complejidad teórica no cambia con N — es atribuible a la sobrecarga creciente del *range partitioning* y del *shuffle* del motor.

IV. PROPUESTA DE INTEGRACIÓN AL PFC

IV-A. Caso de uso y arquitectura de integración

El PFC de referencia es SCLI, un sistema de micro-servicios (Java 21, Spring Boot 4.1.0, Spring Cloud Gate-

way, CockroachDB, Docker Compose) que gestiona usuarios, laboratorios, académico y reservas/solicitudes de laboratorios informáticos institucionales. Actualmente el servicio reservas-solicitudes-service (puerto 8084, base reservas_db) está en fase de integración al gateway.

El caso de uso concreto propuesto es la **generación periódica del ranking de laboratorios y solicitudes de mayor demanda**, aplicación directa de T5: ordenar el historial acumulado de solicitudes_reserva por número de participantes y duración, con fecha e identificador como desempate, y publicar el *Top-N* de mayor demanda para apoyar la asignación de franjas horarias, la priorización de solicitudes en SOLICITUD_APROBAR y la planificación de ampliación de capacidad. Se trata de un proceso por **lotes (batch)**, no en flujo, porque el ranking no necesita recalcularse ante cada solicitud individual, y ejecutarlo con una periodicidad fija (p. ej., diaria o al cierre de cada período de matrícula) es suficiente para las decisiones de negocio que soporta.

La Fig. 2 muestra el punto de inserción propuesto: un nuevo componente ranking-batch-service que se suscribe periódicamente a una réplica de solo lectura de reservas_db (evitando carga adicional sobre la base transaccional), ejecuta el ordenamiento distribuido y publica el *Top-N* resultante de vuelta en reservas_db para que reservas-solicitudes-service lo consuma a través del gateway ya existente en :9090.

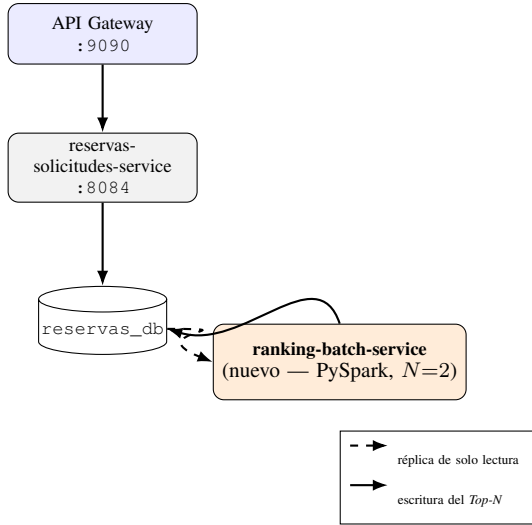


Figura 2: Punto de inserción propuesto de ranking-batch-service en la arquitectura de SCLI. Elaboración propia (TikZ), sobre la topología documentada en el repositorio del PFC.

IV-B. Umbral de rentabilidad y dimensionamiento

Se estimaron los parámetros de la Ec. (5) a partir de las mediciones propias de T5. La tasa secuencial α se estimó directamente como $\alpha = T_{\text{pandas}}(n)/n$ con $n = 500\,000$:

$$\alpha = \frac{1,52521 \text{ s}}{500\,000} = 3,0504 \times 10^{-6} \text{ s/registro.}$$

Los parámetros β (sobrecarga fija de la sesión distribuida) y γ (costo marginal distribuido) se estimaron por **mínimos cuadrados ordinarios** sobre las tres mediciones propias de PySpark en T5, tomando $x = n/N$ como variable independiente (válido porque el modelo de la Ec. (5) depende de n y N exclusivamente a través de esa razón; al mantenerse n fijo en 500 000 y variar $N \in \{1, 2, 4\}$, se obtienen tres pares (x, T_{dist}) equivalentes a un barrido de n/N):

$$\beta = 1,1615 \text{ s}, \quad \gamma = 6,9545 \times 10^{-7} \text{ s/registro.}$$

Para $N = 2$ (la configuración de mejor eficiencia observada, Sección V), la Ec. (5) exige $\alpha > \gamma/N$; en efecto $3,0504 \times 10^{-6} > 3,4773 \times 10^{-7}$, por lo que el umbral es válido:

$$n_{N=2}^* = \frac{\beta}{\alpha - \gamma/2} = \frac{1,1615}{2,7027 \times 10^{-6}} \approx 429\,751 \text{ registros.}$$

Para $N = 4$, análogamente, $n_{N=4}^* \approx 403\,777$ registros.

Este es el hallazgo más relevante de la sección: a diferencia de una transformación ligera como T3 (cuyo umbral, calculado en un análisis paralelo con la misma metodología, ronda los 2,68 millones de registros), **el umbral de rentabilidad de T5 — entre 404 000 y 430 000 registros según N — ya está por debajo del volumen actual del experimento (500 000)**. Es decir, para una operación de ordenamiento global costosa como la generación del ranking de demanda, PySpark ya es la opción más barata al volumen que SCLI manejaría desde el primer período de operación, sin necesidad de esperar a que el sistema crezca.

Modelo de servicio en la nube. Dado que SCLI ya despliega mediante Docker Compose, se propone un modelo de **contenedores gestionados bajo demanda** (p. ej., un *job* de Spark en modo *cluster* lanzado por un orquestador como Kubernetes CronJob o, en un proveedor gestionado, AWS Glue/EMR en modo transitorio), creado únicamente durante la ventana de ejecución diaria del ranking y destruido después [7]. Se recomienda $N = 2$ *executors* y no $N = 4$: además de tener el umbral de rentabilidad más favorable, $N = 2$ ofrece la mejor eficiencia observada (Sección V), por lo que asignar el doble de recursos en producción desperdiciaría presupuesto sin mejorar — de hecho, empeorando — el tiempo de respuesta.

Riesgos identificados: (i) el ajuste de β, γ se realizó con solo tres observaciones a un único valor de n , por lo que n^* es una extrapolación y debería recalibrarse con mediciones a distintos volúmenes reales antes de fijar una decisión de infraestructura; (ii) el comportamiento no monótono observado en $N = 4$ sugiere que la configuración óptima real es $N = 2$ y no el máximo disponible, hallazgo que debe confirmarse en producción con el volumen real de SCLI; (iii) el uso de una réplica de solo lectura introduce un desfase (*staleness*) entre el estado transaccional y el ranking publicado, aceptable para una periodicidad diaria pero no para decisiones en tiempo real.

IV-C. Postura crítica y alternativas descartadas

¿Es Spark la opción correcta para SCLI hoy? La respuesta depende de *qué* proceso se distribuya, y este informe aporta evidencia cuantitativa para responder de forma diferenciada,

no categórica: para las transformaciones ligeras del *pipeline* (filtrado, agrupación, *join* simple), pandas fue más rápido que PySpark con $N = 4$ y su umbral de rentabilidad está muy por encima del volumen operacional realista de SCLI en sus primeros años. Pero para **T5, el umbral ya se cruzó al volumen actual**: la postura sostenida en los propios datos es que SCLI **sí** debería adoptar PySpark, específicamente para el servicio de generación de rankings, sin esperar a que el sistema crezca.

Se evaluaron y descartan dos alternativas para este caso de uso específico:

- **Ordenamiento vía SQL (`ORDER BY ... LIMIT`) directamente en CockroachDB.** CockroachDB soporta ordenamiento distribuido nativo sobre su propio motor SQL. Para el volumen actual de SCLI, esta opción evita introducir un motor adicional y reutiliza la infraestructura ya desplegada; se descarta como alternativa *definitiva* porque el ranking de demanda de este caso de uso requiere combinar múltiples criterios de desempate y, previsiblemente, lógica de puntuación adicional (similar a T4) que un `ORDER BY` plano no expresa con la misma flexibilidad que un *pipeline* de DataFrames, y porque acoplar cómputo analítico pesado a la base transaccional compite por los mismos recursos que las escrituras operacionales de reservas-solicitudes-service.
- **Ordenamiento en pandas puro, programado periódicamente.** Dado que el umbral n^* ya se cruzó, esta alternativa es *más lenta* que PySpark al volumen actual (1,5252s frente a 1,1733s con $N = 2$) y su ventaja de simplicidad no compensa la pérdida de tiempo medida; se descarta como opción de producción para este caso de uso específico, aunque sigue siendo la opción correcta para las transformaciones ligeras del resto del *pipeline* (T1–T3), donde el umbral no se ha cruzado.

En consecuencia, la recomendación es **adoptar PySpark con $N = 2$ *executors*** para `ranking-batch-service` desde la fase actual de SCLI, manteniendo pandas para el resto de procesos analíticos ligeros del sistema — una arquitectura híbrida, no una migración total, decidida transformación por transformación según el umbral n^* propio de cada una.

V. FORMALIZACIÓN MATEMÁTICA

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\max} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p}. \quad (2)$$

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (3)$$

$$E(N) = \frac{S(N)}{N}. \quad (4)$$

$$n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \text{válido si } \alpha > \frac{\gamma}{N}. \quad (5)$$

Con los datos de T5: $E(2) = S(2)/2 = 0,6662$ y $E(4) = S(4)/4 = 0,2881$ (Ec. (4)): la eficiencia cae a menos de la mitad al duplicar de nuevo el número de *executors*, coherente con la sobrecarga creciente diagnosticada en la Sección III y con la recomendación de $N = 2$ de la Sección IV.

VI. CONCLUSIONES

1. Para T5 (ordenamiento global y *Top-20* de 500 000 solicitudes), el *speedup* experimental interno no es monótono en N : $S(2) = 1,3324$ supera a $S(4) = 1,1524$, y el primero excede en un 21,5% la predicción de la Ley de Amdahl ajustada en $N = 4$, con un límite máximo teórico $S_{\max} = 1,2141$ notablemente más bajo que el de una transformación como T3 — evidencia de que T5 tiene una fracción paralelizable estructuralmente pequeña por la fusión final del *Top-20* en el *driver* (Sección II).
2. La métrica de Karp-Flatt creció de $e(2) = 0,5010$ a $e(4) = 0,8236$, con un punto de partida ya alto en $N = 2$, lo que atribuye la pérdida de eficiencia tanto a una fracción serial algorítmica genuina (la recolección final del *Top-20*) como a la sobrecarga creciente del *range partitioning* del motor al escalar N (Sección III).
3. El umbral de rentabilidad calculado para SCLI es $n^* \approx 404\,000$ – $430\,000$ registros según N , **ya por debajo del volumen actual de 500 000**; a diferencia de las transformaciones ligeras del *pipeline*, se recomienda adoptar PySpark con $N = 2$ *executors* para `ranking-batch-service` desde ahora, sin esperar a que el sistema crezca (Sección IV).

DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

Para la elaboración del presente informe se utilizó una herramienta de inteligencia artificial como apoyo en: (i) la organización de ideas, la corrección de ambigüedades y la revisión gramatical del documento; (ii) la explicación de conceptos relacionados con Análisis de Rendimiento Paralelo y temas afines; y (iii) la orientación en la comparación de alternativas entre pandas y spark y los análisis ejecutados. La selección de las fuentes bibliográficas, la revisión de su pertinencia y la verificación de las referencias fueron realizadas manualmente por el autor. Cada cita utilizada en el informe fue comprobada de manera individual mediante la revisión de sus metadatos, DOI y contenido académico correspondiente. Para la consulta de los artículos científicos se utilizaron repositorios académicos y plataformas de acceso a publicaciones científicas, entre ellas ResearchGate, Sci-Hub y otros servicios de consulta disponibles para fines de revisión bibliográfica. El análisis, la interpretación de los resultados y la postura crítica presentada en el informe son de autoría individual del estudiante, quien revisó y editó el contenido en su totalidad antes de la entrega.

REFERENCIAS

- [1] M. Zaharia et al., “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, n.º 11, págs. 56-65, 2016. DOI: 10.1145/2934664.
- [2] M. D. Hill y M. R. Marty, “Amdahl’s Law in the Multicore Era,” *Computer*, vol. 41, n.º 7, págs. 33-38, 2008. DOI: 10.1109/MC.2008.209.
- [3] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” en *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference (AFIPS ’67 (Spring))*, Atlantic City, NJ, USA: ACM, 1967, págs. 483-485. DOI: 10.1145/1465482.1465560.
- [4] J. L. Gustafson, “Reevaluating Amdahl’s Law,” *Communications of the ACM*, vol. 31, n.º 5, págs. 532-533, 1988. DOI: 10.1145/42411.42415.
- [5] A. H. Karp y H. P. Flatt, “Measuring Parallel Processor Performance,” *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: 10.1145/78607.78614.
- [6] M. Zaharia et al., “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” en *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, 2012, págs. 15-28, ISBN: 978-931971-92-8.
- [7] J. Dean, *Big Data, Data Mining, and Machine Learning: Value Creation for Business Leaders and Practitioners*. Hoboken, NJ, USA: Wiley, 2014, ISBN: 978-1-118-61804-2.